

# Deep Learning for NLP – Homework 2

## Response Clarity Classification (CLARITY)

Student name: Αλέξανδρος Γκιόφης  
ID: sdi2200284

---

Course: *Artificial Intelligence II (YS19 M226 M262 M325 M405)*  
Semester: *Spring Semester 2025-2026*

### Contents

|          |                                                                             |          |
|----------|-----------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Abstract</b>                                                             | <b>4</b> |
| <b>2</b> | <b>Εισαγωγή στο <b>task</b> και τι είχα να κάνω</b>                         | <b>4</b> |
| 2.1      | Τι είναι το πρόβλημα . . . . .                                              | 4        |
| 2.2      | Τι έπρεπε να φτιάξω, σύμφωνα με το assignment . . . . .                     | 5        |
| 2.3      | Kaggle notebooks και leaderboard . . . . .                                  | 5        |
| 2.4      | Τι είναι τέλος πάντων ένα Transformer (για να μη μένουν εκκρεμότητες) . . . | 5        |
| <b>3</b> | <b>Data processing and analysis</b>                                         | <b>6</b> |
| 3.1      | Pre-processing . . . . .                                                    | 6        |
| 3.1.1    | Word normalization (πιο εκλεπτυσμένη ιδέα) . . . . .                        | 7        |
| 3.2      | Τι είδα στα δεδομένα . . . . .                                              | 7        |
| 3.2.1    | Class imbalance . . . . .                                                   | 7        |
| 3.2.2    | Η “γκρίζα” Ambivalent κλάση . . . . .                                       | 7        |
| 3.3      | Train / validation split . . . . .                                          | 8        |
| 3.4      | Vectorization (πώς γίνεται κείμενο σε αριθμούς) . . . . .                   | 8        |

|        |                                                                                           |    |
|--------|-------------------------------------------------------------------------------------------|----|
| 3.4.1  | Παλιό style: TF-IDF . . . . .                                                             | 8  |
| 3.4.2  | Νέο style: Transformer contextual embeddings . . . . .                                    | 8  |
| 3.4.3  | Πώς ταίζω την ερώτηση και την απάντηση μαζί . . . . .                                     | 9  |
| 4      | Τα μοντέλα που δοκίμασα – σύντομος οδηγός                                                 | 9  |
| 4.1    | BERT (bert-base-uncased) . . . . .                                                        | 9  |
| 4.2    | DistilBERT (distilbert-base-uncased) . . . . .                                            | 9  |
| 4.3    | DeBERTa-v3-base (microsoft/deberta-v3-base) . . . . .                                     | 10 |
| 5      | <b>Algorithms and Experiments</b>                                                         | 10 |
| 5.1    | Συνολική στρατηγική . . . . .                                                             | 10 |
| 5.2    | Πειράματα – κύριος πίνακας . . . . .                                                      | 11 |
| 5.3    | Λεπτομέρειες ανά πείραμα – τι δοκίμασα και τι έμαθα . . . . .                             | 12 |
| 5.3.1  | DistilBERT: ο φτηνός γρήγορος εξερευνητής . . . . .                                       | 12 |
| 5.3.2  | Από DistilBERT σε BERT: μικρή βελτίωση . . . . .                                          | 13 |
| 5.3.3  | DeBERTa baseline: το πραγματικό άλμα . . . . .                                            | 13 |
| 5.3.4  | Ιδέα 1: Feature tokens μέσα στο κείμενο (απέτυχε) . . . . .                               | 13 |
| 5.3.5  | Ιδέα 2: Numeric side branch (το breakthrough) . . . . .                                   | 14 |
| 5.3.6  | Ιδέα 3: Word normalization (ίσος συν) . . . . .                                           | 14 |
| 5.3.7  | Ιδέα 4: Focused answer view (διαφορετικός τρόπος να “διαβάσει”<br>την απάντηση) . . . . . | 14 |
| 5.3.8  | Ιδέα 5: Phase 18 alpha ensemble (το καλύτερο local val) . . . . .                         | 15 |
| 5.3.9  | Ιδέα 6: Dual-view architecture (απέτυχε) . . . . .                                        | 15 |
| 5.3.10 | Ιδέα 7: Evasion-label stacking (απέτυχε) . . . . .                                        | 16 |
| 5.3.11 | Ιδέα 8: Περισσότερα epochs (απέτυχε) . . . . .                                            | 17 |
| 5.3.12 | Ιδέα 9: Longer context (max_length=512) στο DeBERTa (απέτυχε) .                           | 17 |

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| 5.4      | Hyper-parameter tuning . . . . .                              | 17        |
| 5.5      | Optimization techniques . . . . .                             | 17        |
| 5.6      | Evaluation . . . . .                                          | 18        |
| 5.6.1    | Γιατί macro-F1 και όχι plain accuracy . . . . .               | 18        |
| 5.6.2    | ROC curve . . . . .                                           | 19        |
| 5.6.3    | Learning curves . . . . .                                     | 19        |
| 5.6.4    | Confusion matrices . . . . .                                  | 20        |
| 5.7      | Subgroup analysis . . . . .                                   | 20        |
| <b>6</b> | <b>Detailed error analysis</b>                                | <b>21</b> |
| 6.1      | Ποια κλάση είναι η πιο δύσκολη . . . . .                      | 21        |
| 6.2      | Συγκεκριμένα παραδείγματα λαθών . . . . .                     | 21        |
| 6.3      | Patterns στα λάθη . . . . .                                   | 22        |
| 6.4      | Διαφέρουν τα στρονγέρ μοντέλα στα λάθη τους; . . . . .        | 22        |
| 6.5      | Τι θα έκανε ένα “ιδανικό” μοντέλο για αυτό το task; . . . . . | 23        |
| <b>7</b> | <b>Results and Overall Analysis</b>                           | <b>23</b> |
| 7.1      | Κύρια ευρήματα . . . . .                                      | 23        |
| 7.2      | Best trial: αρχιτεκτονική . . . . .                           | 24        |
| 7.3      | Comparison with Assignment 1 . . . . .                        | 24        |
| 7.4      | Modifications που δοκίμασα και απέτυχαν – σύνοψη . . . . .    | 25        |
| 7.5      | Τι θα έκανα διαφορετικά αν είχα περισσότερο χρόνο . . . . .   | 26        |
| <b>8</b> | <b>Συμπεράσματα</b>                                           | <b>27</b> |
| <b>9</b> | <b>Bibliography</b>                                           | <b>28</b> |

## 1. Abstract

Σε αυτή την εργασία είχαμε ένα **dataset** (το CLARITY) με ζευγάρια ερώτηση-απάντηση από πολιτικούς και έπρεπε να φτιάξουμε ένα μοντέλο που, για κάθε ζευγάρι, θα προβλέπει σε ποια από τις τρεις κατηγορίες ανήκει η απάντηση: *Clear Reply* (ο πολιτικός όντως απαντάει), *Clear Non-Reply* (ξεφεύγει εντελώς), ή *Ambivalent* (η απάντηση είναι κάπου στη μέση – λέει κάτι αλλά υπεκφεύγει κιόλας). Στην πρώτη εργασία είχα χρησιμοποιήσει κλασικές μεθόδους τύπου TF-IDF + Logistic Regression. Εδώ έπρεπε να γυρίσω σε **deep learning** και συγκεκριμένα σε Transformer-based μοντέλα.

Ξεκίνησα σταδιακά: πρώτα DistilBERT [4] γιατί είναι το πιο ελαφρύ και πρόλαβα να κάνω πολλά πειράματα γρήγορα, μετά BERT [1] σαν **standard reference**, και τέλος DeBERTa-v3-base [2] που είναι το πιο δυνατό από τα τρία και ήταν ο τελικός μου “πρωταγωνιστής”. Πάνω στο DeBERTa πειραματίστηκα με αρκετά τρικ – άλλα δούλεψαν, αρκετά όχι, και νομίζω ότι από τις αποτυχίες έμαθα περισσότερα από ό,τι από τις επιτυχίες.

Το καλύτερο public Kaggle αποτέλεσμα ήταν 0.70 macro-F1 στο public leaderboard με username **AlexTuring - sdi2200284**. Στο τοπικό μου validation set, το καλύτερο score έφτασε 0.7136 macro-F1, με ensemble τριών διαφορετικών εκδοχών του ίδιου DeBERTa μοντέλου. Το ότι το Kaggle πέφτει λίγο σε σχέση με το local val είναι αναμενόμενο – δεν είναι ακριβώς ίδιο **data distribution**, οπότε αυτό μου έδωσε και μια ρεαλιστική ιδέα του πόσο γενικεύει το μοντέλο μου.

## 2. Εισαγωγή στο **task** και τι είχα να κάνω

### 2.1. Τι είναι το πρόβλημα

Το CLARITY task ακούγεται απλό αλλά δεν είναι. Φαντάσου ότι ένας δημοσιογράφος ρωτάει έναν πρόεδρο: “*Why are you not having a joint press conference with Putin?*”. Ο πρόεδρος μπορεί:

- να απαντήσει ευθέως (*Clear Reply*),
- να αλλάξει εντελώς θέμα (*Clear Non-Reply*),
- ή – και αυτό είναι το συχνότερο – να πει κάτι που φαίνεται απάντηση αλλά στην ουσία είναι μισή ή ξεγλιστράει (*Ambivalent*).

Το δύσκολο είναι ότι οι πολιτικές απαντήσεις σπάνια είναι ή τελείως **clear** ή τελείως **evasion**. Συχνά ένας πολιτικός θα πει 5 προτάσεις γενικότητες και μετά μία πρόταση που πραγματικά απαντάει, ή θα δώσει **context** και αιτιολογήσεις και μετά θα “ξεχάσει” να καταλήξει. Η *Ambivalent* κατηγορία είναι ουσιαστικά η “γκρίζα ζώνη”. Αυτό σημαίνει ότι το μοντέλο δεν μπορεί απλώς να ψάξει **keywords**. Πρέπει να καταλάβει αν η απάντηση όντως απαντάει στην ερώτηση – δηλαδή πρέπει να συνδέσει το **semantic content** της απάντησης με αυτό που ζητήθηκε.

## 2.2. Τι έπρεπε να φτιάξω, σύμφωνα με το **assignment**

Σύμφωνα με τις οδηγίες του Homework 2, έπρεπε:

1. να κάνω **fine-tuning** σε τρία συγκεκριμένα pretrained Transformer μοντέλα: `bert-base-uncased`, `distilbert-base-uncased`, και `microsoft/deberta-v3-base`,
2. να γράψω δικό μου **training loop** σε PyTorch (απαγορευόταν το Hugging Face Trainer API),
3. να αποφασίσω εγώ πώς θα φτιάξω το **input**, πόσα **epochs**, ποιον **optimizer**, **learning rate**, **max length** κ.λπ.,
4. να συγκρίνω τα τρία μοντέλα μεταξύ τους, να αναλύσω τα λάθη τους, και να δω αν διαφορετικά μεγέθη μοντέλων αποτυγχάνουν με διαφορετικό τρόπο,
5. να υποβάλω 3 ξεχωριστά **Kaggle notebooks** (ένα ανά μοντέλο),
6. να γράψω αυτό το **report**.

## 2.3. Kaggle notebooks και leaderboard

Τα τρία τελικά Kaggle notebooks που αντιστοιχούν στα απαιτούμενα μοντέλα είναι:

- BERT: [final-bert](#)
- DeBERTa: [final-deberta](#)
- DistilBERT: [final-distilbert](#)

Στο **public leaderboard** εμφανίζομαι ως **AlexTuring - sdi2200284**. Το public αποτέλεσμα που κρατάω ως τελικό για την αναφορά είναι **0.70 macro-F1**.

Για να μη μπερδευτεί αυτό με τα πολλά DeBERTa πειράματα: το τελικό Kaggle submission ήταν το **Phase 12 DeBERTa + numeric side branch alpha ensemble**. Δηλαδή raw full question–answer input, τα 19 numeric features στο side branch, seeds 42/0/1 και fixed weights [0.9, 0.0, 0.1]. Το Phase 18 είχε καλύτερο local validation score, αλλά δεν ήταν αυτό που επέλεξα για το final Kaggle submission.

## 2.4. Τι είναι τέλος πάντων ένα **Transformer** (για να μη μένουν εκκρεμότητες)

Πριν προχωρήσω στα πειράματα νομίζω ότι αξίζει να κάνω μια σύντομη εξήγηση των βασικών εννοιών, γιατί μετά αναφέρομαι σε αυτές συνέχεια. Αν ο αναγνώστης τα ξέρει ήδη, μπορεί να προσπεράσει.

Ένα **Transformer** μοντέλο είναι ένα νευρωνικό δίκτυο που επεξεργάζεται ακολουθίες (όπως κείμενα), και η βασική του “μαγική” δουλειά είναι ένα στρώμα που λέγεται **self-attention**. Με απλά λόγια, για κάθε λέξη στο κείμενο το μοντέλο υπολογίζει “πόσο σχετική είναι κάθε άλλη λέξη με αυτή τη λέξη”, και χρησιμοποιεί αυτή την πληροφορία για να φτιάξει μια αναπαράσταση της λέξης που να εξαρτάται από το **context**. Αυτό είναι σημαντικό γιατί η ίδια λέξη μπορεί να σημαίνει διαφορετικά πράγματα ανάλογα με τι υπάρχει γύρω της (“bank” σε bank account vs river bank).

**Pretrained** σημαίνει ότι το μοντέλο έχει ήδη εκπαιδευτεί σε τεράστιες ποσότητες κειμένου από το **internet**, με **tasks** σαν “μάντεψε ποια λέξη λείπει”. Έτσι έχει μάθει **general** γλωσσικά **patterns** πριν δει καν το δικό μας **dataset**. **Fine-tuning** σημαίνει ότι παίρνουμε αυτό το έτοιμο μοντέλο, βάζουμε από πάνω ένα μικρό **classifier** (κάτι σαν τελικό **linear layer** που βγάζει 3 **logits**, ένα ανά κλάση), και συνεχίζουμε την εκπαίδευση στα δικά μας πιο λίγα δεδομένα. Έτσι “προσαρμόζεται” το μοντέλο στο **specific task**.

**Tokenizer**: τα **Transformer** δεν δουλεύουν με ολόκληρες λέξεις, τις κόβουν σε υπολέξεις (subwords) με βάση μια τεχνική σαν το **BPE** ή το **SentencePiece**. Παράδειγμα: η λέξη “unhappiness” μπορεί να γίνει [“un”, “happi”, “ness”]. Έτσι το μοντέλο δεν χάνεται όταν συναντάει λέξη που δεν έχει ξαναδεί – την σπάει σε γνωστά κομμάτια.

**Embedding**: ένα διάνυσμα από αριθμούς που αναπαριστά μια λέξη ή ολόκληρο πρόταση. Π.χ. το **[CLS] token** (ένα ειδικό **token** που μπαίνει στην αρχή κάθε **input** στο **BERT**) μετά το **pass** από το μοντέλο γίνεται ένα διάνυσμα **768** διαστάσεων που “περιέχει” περιεκτικά την πληροφορία όλης της πρότασης – και πάνω σε αυτό βάζουμε τον **classifier** για να βγάλουμε τις 3 κλάσεις.

### 3. Data processing and analysis

#### 3.1. Pre-processing

Το αρχικό **training set** είχε 3448 γραμμές. Έκανα ένα σχετικά συντηρητικό **cleaning** και έμειναν 3412 παραδείγματα. Συγκεκριμένα:

1. Αφαίρεση **contradictory duplicates**: είχε μερικά ζευγάρια **Q-A** που εμφανίζονταν δύο ή περισσότερες φορές με διαφορετικό **label**. Αυτό για το μοντέλο είναι θόρυβος – το ίδιο **input** δεν μπορεί να έχει δύο διαφορετικά **labels** και να βγάλει νόημα. Τα έβγαλα.
2. Αφαίρεση κάποιων προβληματικών **rows** που είχα εντοπίσει στην πρώτη εργασία (πολύ μικρό **text**, **OOV-heavy** κλπ).

Το βασικό μου **axiom** από εκεί και πέρα ήταν: δεν κάνω **heavy cleaning** του φυσικού κειμένου. Τα **Transformer** είναι ήδη **pretrained** σε **huge** ποσότητες ακατέργαστου **text**, οπότε δεν θέλουν – και πιθανόν χάνουν – αν αρχίσω να αφαιρώ **punctuation**, να **lowercase**άρω επιθετικά, να βγάλω **stopwords** κλπ. Επιπλέον στο συγκεκριμένο **task** μικρά **linguistic** σημάδια όπως “well, ...”, “I think...”, “let me be clear” είναι ακριβώς ο τύπος της πληροφορίας που ξεχωρίζει μια **clear** απάντηση από μια **ambivalent**. Αν τα έσβηνα, θα έσβηνα σήμα.

**3.1.1. Word normalization** (πιο εκλεπτυσμένη ιδέα). Δοκίμασα όμως μια πιο στοχευμένη μορφή **normalization**, η οποία τελικά βοήθησε λίγο. Η ιδέα: σε αυτό το **task** δεν μας ενδιαφέρει αν ο πολιτικός είπε “in 1981” ή “in 1979”. Ενδιαφέρει το ότι αναφέρθηκε σε χρονιά. Οπότε αντικατέστησα **patterns** σαν χρονιές, ποσά, ποσοστά, ημερομηνίες με πραγματικές αγγλικές λέξεις: **year, money, percent, date**.

Σημαντικό: έβαλα κανονικές αγγλικές λέξεις, όχι τεχνητά **tokens** όπως [A\_YEAR]. Η διαφορά είναι κρίσιμη: το DeBERTa ξέρει τι σημαίνει **year**, έχει δει τη λέξη χιλιάδες φορές στο **pretraining**. Αν όμως του βάλω [A\_YEAR] σαν νέο **token**, πρέπει να μάθει **embedding** γι’ αυτό από την αρχή με μόνο 3000 παραδείγματα. Είναι σαν να του βάζω **hieroglyphs**. Ξανασυναντάμε αυτή τη διάκριση παρακάτω.

### 3.2. Τι είδα στα δεδομένα

**3.2.1. Class imbalance.** Οι τρεις κλάσεις δεν είναι ισόποσες. Η **Ambivalent** είναι η μεγαλύτερη (περίπου 60%), η **Clear Reply** είναι μεσαία (περίπου 30%), και η **Clear Non-Reply** είναι η μικρότερη (μόλις περίπου 10%). Αυτή η ανισορροπία επηρεάζει τη μετρική: αν δώναμε περισσότερο βάρος σε **accuracy**, θα φτιάχναμε ένα μοντέλο που λέει συνέχεια “**Ambivalent**” και θα είχε αξιοπρεπές **score**. Γι’ αυτό το **macro-F1** είναι η σωστή μετρική – δίνει ίσο βάρος και στις τρεις κλάσεις.

**3.2.2. Η “γκρίζα” *Ambivalent* κλάση.** Από τα διαγράμματα και την ανάλυση που έκανα, η **Ambivalent** ήταν η πιο μπερδευτική κατηγορία. Μοιάζει σαν μαγνήτης – όταν το μοντέλο έχει αμφιβολία αν μια απάντηση είναι **Clear** ή **Non-Reply**, την πετάει στο **Ambivalent**. Λογικό, αν το σκεφτείς: αν ένα μοντέλο δεν είναι σίγουρο για το ποιο **extreme** να επιλέξει, η “μέση” κατηγορία είναι ασφαλής **choice** από άποψη **loss**.

Αυτό φαίνεται καθαρά στο τελικό **confusion matrix** του καλύτερου DeBERTa+numfeat single μοντέλου:

$$C = \begin{bmatrix} 69 & 34 & 2 \\ 34 & 155 & 13 \\ 1 & 9 & 25 \end{bmatrix}$$

Οι γραμμές είναι το πραγματικό label (CR / AMB / CNR), οι στήλες είναι η πρόβλεψη. Διαβάζοντάς το:

- Από 105 πραγματικά **Clear Reply**, 34 πήγαν λάθος ως **Ambivalent**.
- Από 35 πραγματικά **Clear Non-Reply**, 9 πήγαν λάθος ως **Ambivalent**.
- Πολύ λίγα “αχραία” λάθη: μόλις 2 **CR** ταξινομήθηκαν ως **CNR** και 1 **CNR** ως **CR**. Δηλαδή το μοντέλο σπάνια κάνει το “φοβερό” λάθος του να μπερδέψει εντελώς τις δύο **extreme** κατηγορίες. Τα λάθη είναι σχεδόν πάντα “slip το ένα τικ” – από **CR** ή **CNR** στο γειτονικό **Ambivalent**.

### 3.3. Train / validation split

Χώρισα τα 3412 cleaned examples σε:

- train: 3070 (90%)
- validation: 342 (10%)

με **stratification** ως προς το **label**. **Stratification** σημαίνει ότι, αντί να πετάζουμε τυχαία 10% στο val και να ελπίζουμε ότι θα είναι ισορροπημένο, αναγκάζουμε το **split** να κρατάει την αναλογία των κλάσεων ίδια με το αρχικό **set**. Αυτό είναι ιδιαίτερα σημαντικό όταν έχουμε μικρή κλάση (**Clear Non-Reply**): ένα τυχαίο **split** θα μπορούσε εύκολα να κρατήσει μόλις 20 ή 50 παραδείγματα CNR στο val, και το **macro-F1** θα ήταν πολύ θορυβώδες.

Δεν έκανα **cross-validation**. Αρχικά γιατί δεν είχα όρεξη να χορεύω 5x χρόνο εκπαίδευσης για κάθε πείραμα, και κυρίως γιατί στα **Transformer** η μεγαλύτερη πηγή **variance** δεν είναι το **split** αλλά το **seed** (το αρχικό αρχικοποιημένο **classifier head** + **dropout randomness**). Οπότε για **important runs** δοκίμασα μερικά διαφορετικά **seeds** (συνήθως 42, 0, 1) πάνω στο ίδιο **fixed split**. Αυτό δίνει πιο καθαρή εικόνα, γιατί κρατάει το **split** σταθερό σε όλα τα μοντέλα.

### 3.4. Vectorization (πώς γίνεται κείμενο σε αριθμούς)

**3.4.1. Παλιό style: TF-IDF.** Στην πρώτη εργασία είχα χρησιμοποιήσει TF-IDF: κάθε κείμενο γίνεται ένα τεράστιο **sparse** διάνυσμα όπου κάθε διάσταση αντιστοιχεί σε μία λέξη ή **n-gram**, και η τιμή είναι μια συνάρτηση του πόσο συχνά εμφανίζεται η λέξη στο κείμενο πολλαπλασιασμένη με ένα **weight** που πέφτει αν η λέξη είναι **common** σε όλο το **corpus** (**inverse document frequency**). Λειτουργεί ικανοποιητικά για **baseline** αλλά:

- αγνοεί τη σειρά των λέξεων (“δεν θα απαντήσω” = “απαντήσω δεν θα” στο TF-IDF),
- δεν καταλαβαίνει συνώνυμα ή παραφράσεις,
- δεν “δεν συγκρίνει” απάντηση με ερώτηση – κάθε ένα είναι ξεχωριστό **bag of words**.

**3.4.2. Νέο style: Transformer contextual embeddings.** Με τα **Transformer** η αναπαράσταση είναι ριζικά διαφορετική. Ο **tokenizer** σπάει το κείμενο σε **subwords**, το μοντέλο τα περνάει από πολλά **layers self-attention**, και βγαίνουν **contextual embeddings** – δηλαδή το διάνυσμα της λέξης “clear” στη φράση “let me be clear” δεν είναι το ίδιο με το “a clear answer”. Αυτό αλλάζει το παιχνίδι, γιατί το μοντέλο μπορεί να καταλάβει **discourse markers**, **hedging**, **irony** κ.λπ.

**3.4.3.** Πώς ταίζω την ερώτηση και την απάντηση μαζί. Το **input** δεν ήταν μόνο η απάντηση. Το βασικό μου σχήμα ήταν **two-segment input**, δηλαδή:

```
[CLS] question_text [SEP] answer_text [SEP]
```

Αυτό είναι ο **standard** τρόπος να δώσεις ζευγάρι κειμένων σε ένα **BERT-family** μοντέλο. Το μοντέλο μαθαίνει **token type IDs** (0 για το πρώτο **segment**, 1 για το δεύτερο) ώστε να ξέρει ποιο είναι ποιο, και η **self-attention** μπορεί να “κοιτάει” και τα δύο μαζί για να φτιάξει το τελικό **sentence embedding**. Δοκίμασα και **concat\_sep** (απλό **concatenation** με **[SEP]**) στο **DistilBERT** – για το **DistilBERT** τα δύο σχήματα έβγαζαν ίδιες προβλέψεις γιατί το **DistilBERT** δεν έχει **token\_type\_ids embedding** (έχει αφαιρεθεί στο **distillation**). Στο **BERT/DeBERTa** το **two\_segment** είναι το **natural choice** και έμεινα σε αυτό.

## 4. Τα μοντέλα που δοκίμασα – σύντομος οδηγός

Σε αυτό το **section** εξηγώ με δύο λόγια τι είναι το καθένα από τα τρία μοντέλα που χρησιμοποίησα. Νομίζω χρειάζεται γιατί δεν είναι όλα ισοδύναμα – το καθένα έχει διαφορετική αρχιτεκτονική και διαφορετικά τρικ.

### 4.1. BERT (bert-base-uncased)

Το αυθεντικό **BERT** [1]. Είναι ένα **Transformer encoder** με 12 layers, hidden size 768 και 110M parameters. “Uncased” σημαίνει ότι το **vocabulary** του δεν διαχωρίζει κεφαλαία/μικρά – το “Hello” και το “hello” γίνονται το ίδιο **token**. **Pretrained** με δύο **tasks**: **Masked Language Modeling** (μάντεψε σβησμένες λέξεις) και **Next Sentence Prediction** (αν δύο προτάσεις είναι διαδοχικές). Το βασικό **feature** που κληρονόμησαν όλα τα παρόμοια μοντέλα είναι το **bidirectional self-attention** – κάθε **token** κοιτάει και αριστερά και δεξιά μέσα στο **sentence** ταυτόχρονα, αντί να διαβάζει **left-to-right** όπως κάνουν τα **GPT-style** μοντέλα.

### 4.2. DistilBERT (distilbert-base-uncased)

Το **DistilBERT** [4] είναι μια “μαθήτριά” εκδοχή του **BERT**. Ο τρόπος που το έφτιαξαν λέγεται **knowledge distillation**: παίρνεις ένα μεγάλο **pretrained** μοντέλο (το “teacher”, εδώ **BERT**) και εκπαιδεύεις ένα μικρότερο (το “student”) όχι απλώς να μαντέψει τα **labels**, αλλά να μιμηθεί τις **probabilities** του **teacher**. Έτσι το **student** τα καταφέρνει με σχεδόν τη μισή ποσότητα παραμέτρων (66M, 6 layers αντί 12), τρέχει σχεδόν διπλάσια γρήγορα, και χάνει μόνο γύρω στο 3% accuracy σε **generic tasks**.

Επιπλέον το **DistilBERT** αφαιρεί το **NSP loss** και τα **token\_type\_ids embedding**. Πρακτικό **consequence**: στο **dataset** μας το **DistilBERT** δεν χρησιμοποιεί την πληροφορία “ποιο **segment** είναι ερώτηση και ποιο απάντηση”. Όλα τα **tokens** είναι ίδιο **segment** από την οπτική του μοντέλου.

Επέλεξα το DistilBERT σαν το μικρότερο και πιο γρήγορο μοντέλο, για να κάνω **fast iteration** στα πειράματα – αν μια ιδέα δεν δούλευε στο DistilBERT, σπάνια άξιζε να σπαταλήσω χρόνο σε BERT/DeBERTa.

#### 4.3. DeBERTa-v3-base (microsoft/deberta-v3-base)

Το DeBERTa [2] – ή πληρέστερα “Decoding-enhanced BERT with disentangled Attention” – είναι σαν BERT αλλά κάνει δύο σημαντικά πράγματα διαφορετικά:

1. **Disentangled attention.** Στο BERT, το embedding κάθε λέξης είναι (γενικά) **embedding** του ID της λέξης + **positional embedding** (πληροφορία θέσης), και αυτό το **mash** είναι που μπαίνει στη **self-attention**. Στο DeBERTa, η **self-attention** υπολογίζει ξεχωριστά το “**content-to-content attention**” και το “**content-to-position attention**”. Με απλά λόγια: ο Pearson μάθαινε ένα ενιαίο διάνυσμα (“τι λέει η λέξη + πού βρίσκεται”), ενώ το DeBERTa μάθει δύο κανάλια (“τι λέει η λέξη” και “πού βρίσκεται”) και τα συνδυάζει πιο έξυπνα. Αυτό βοηθάει σε **tasks** όπου η σχετική θέση στοιχείων μετράει.
2. **Enhanced mask decoder.** Στο **pretraining**, αντί να προβλέπει την σβησμένη λέξη μόνο από **context**, χρησιμοποιεί επίσης **absolute positional information**. Δεν έχει **direct** συνέπεια στο **fine-tuning** αλλά δίνει καλύτερα **pretrained weights**.

Ειδικά το **deberta-v3-base** χρησιμοποιεί **ELECTRA-style pretraining objective** (**replaced token detection** αντί **masked language modeling**), που έχει δειχτεί ότι είναι πιο **sample-efficient**. Παρότι έχει ίδιο μέγεθος με το BERT-base (140M params εδώ), συνήθως νικάει το BERT καθαρά σε **classification tasks**. Αυτό το επιβεβαίωσα και εγώ: από 0.6417 **macro-F1** (BERT) στο 0.6962 (DeBERTa single seed=42) – αρκετά μεγάλο **jump**.

Πρακτικό **warning** για το **DeBERTa**: είναι **numerically** πιο **fragile** από το BERT. Η πρώτη φορά που το έτρεξα, το **loss** έβγαινε **NaN** ή το μοντέλο κατέρρεε προβλέποντας μόνο **Ambivalent**. Έπρεπε να καθορίσω συγκεκριμένη έκδοση **transformers==4.44.0**, να βάλω **eps=1e-6** στον **optimizer** και να δώσω **differential learning rate** (το **head** με  $10\times$  μεγαλύτερο **learning rate** από το **backbone**). Αυτό το έμαθα με τον πιο επώδυνο τρόπο – δηλαδή τρέχοντας πολλά αποτυχημένα **runs** μέχρι να βρω τον σωστό συνδυασμό. Το αναφέρω εδώ γιατί νομίζω είναι **useful negative finding**: το DeBERTa είναι καλύτερο μοντέλο αλλά δεν είναι “**plug and play**”.

## 5. Algorithms and Experiments

### 5.1. Συνολική στρατηγική

Η βασική μου ιδέα για τα πειράματα ήταν: ξεκίνα από το πιο μικρό και απλό, σιγά σιγά πήγαινε στο πιο μεγάλο και περίπλοκο, και σε κάθε βήμα κράτα **baselines**. Η λογική είναι ότι

αν ένα “έξυπνο” μοντέλο δεν κερδίζει το απλούστερο **baseline**, τότε κάτι πάει λάθος ή το **trick** που πρόσθεσα δεν αξίζει.

Άρα η πορεία ήταν:

1. HW1 baseline (Logistic Regression + TF-IDF): 0.6017 macro-F1.
2. DistilBERT vanilla: ανέβηκε σε 0.64.
3. BERT vanilla: παρόμοιο 0.64.
4. DeBERTa vanilla: jump στο 0.69.
5. DeBERTa + numeric side branch: best single 0.7007.
6. DeBERTa ensembles διαφορετικών seeds και διαφορετικών views: best 0.7136 local val.

| Trial                                | Περιγραφή                                                | Val macro-F1  | Public Kaggle |
|--------------------------------------|----------------------------------------------------------|---------------|---------------|
| HW1 baseline                         | Logistic Regression + TF-IDF                             | 0.6017        | –             |
| DistilBERT                           | max_length=512, lr $3 \cdot 10^{-5}$ , 3 epochs, seed=42 | 0.6405        | –             |
| BERT                                 | max_length=256, lr $2 \cdot 10^{-5}$ , 5 epochs, seed=1  | 0.6417        | –             |
| DeBERTa                              | two-segment, 3 epochs, seed=42, wd=0                     | 0.6962        | 0.68          |
| DeBERTa $\alpha$ -ensemble (3 seeds) | weighted avg over seeds 42/0/1                           | 0.7070        | 0.68          |
| Feature tokens (failed)              | handcrafted tags injected as text tokens                 | 0.6191        | –             |
| <b>DeBERTa + numeric side branch</b> | DeBERTa pooled vec + handcrafted feats via MLP           | <b>0.7007</b> | <b>0.70</b>   |
| More epochs (failed)                 | ίδια ιδέα με 6 epochs                                    | 0.6876        | 0.67          |
| Rich features (failed)               | πολλά extra HW1-style features                           | 0.6593        | 0.64          |
| Evasion stack (failed)               | evasion-label probabilities ως extra features            | 0.6503        | –             |
| Word normalization                   | αντικατάσταση αριθμών με κανονικές λέξεις                | 0.6965        | –             |
| <b>Phase 18 alpha ensemble</b>       | raw + wordnorm + focused view                            | <b>0.7136</b> | –             |
| Calibrated alpha                     | class multipliers post-hoc                               | 0.7154        | –             |
| Dual-view (failed)                   | shared DeBERTa σε full + focused                         | 0.6888        | –             |

Table 1: Συνολικός πίνακας πειραμάτων. Το final submitted Kaggle score είναι 0.70.

## 5.2. Πειράματα – κύριος πίνακας

### 5.3. Λεπτομέρειες ανά πείραμα – τι δοκίμασα και τι έμαθα

Εδώ θέλω να εξηγήσω αναλυτικά τι ακριβώς ήταν κάθε πείραμα, ειδικά αυτά που είχαν “περίεργα” ονόματα. Δεν θέλω να μένει αβέβαιο τι σημαίνει “focused view” ή “dual-view” ή “evasion stack”.

**5.3.1. DistilBERT:** ο φτηνός γρήγορος εξερευνητής. Στο DistilBERT η σημαντικότερη ανακάλυψη ήταν ότι το `max_length=512` βοηθούσε σαφώς. Όταν είχα `max_length=128` ή `256`, πολλές μεγάλες πολιτικές απαντήσεις κόβονταν στη μέση – και αυτό κατέστρεφε ακριβώς το σήμα που ήθελα. Με `ml=512` όλη η απάντηση χωράει στο context. Best DistilBERT seed=42, lr 3e-5, 3 epochs, ml=512 = **0.6405** macro-F1.

Δοκίμασα επίσης πολλές μικρές παραλλαγές που δεν βοήθησαν, και αναφέρω τις πιο interesting:

- **Class weights** στο **cross-entropy loss**: ιδέα ήταν να δώσω μεγαλύτερο βάρος στα πιο σπάνια classes (CR, CNR) ώστε το μοντέλο να μην επικεντρώνεται μόνο στο Ambivalent. Αποτέλεσμα: 0.5955 (ίδιο config). Το CR βελτιώθηκε λίγο, το AMB έπεσε αρκετά, στο σύνολο macro-F1 χειρότερα.
- **Focal loss** ( $\gamma = 2.0$ ): ένα loss που down-weightάρει εύκολα παραδείγματα ώστε το μοντέλο να επικεντρώνεται στα δύσκολα. Έπεσε στο 0.6284 – πιθανόν γιατί τα “εύκολα” ήταν ήδη το σωστό σήμα και το να τα αγνοεί δεν βοήθησε.
- **cw + focal** συνδυασμός: catastrophe, 0.4383. Το AMB κατέρρευσε από 0.78 σε 0.25 F1. Διπλή penalty στις κυρίαρχες κλάσεις – το μοντέλο τρόμαξε να προβλέψει AMB.
- **Negation markers ([NEG] prefix στα verbs** άρνησης, με spaCy): πιο γλωσσολογικό trick, βάζω ένα marker πριν από κάθε verb σε άρνηση. Ιδέα: τα CNR συχνά έχουν αρνητικά verbs (“I won’t say...”). +0.008 στο DistilBERT (0.6484 vs 0.6405). Μικρό improvement αλλά consistent.
- **Multi-task auxiliary head** (multi-task aux=0.3 με fixed 2-layer head): εδώ το dataset έχει ένα δευτερεύον label, το evasion\_label (πιο fine-grained, 9 κλάσεις: Explicit Reply, Claims Ignorance, Dodging, Clarification, ...). Multi-task σημαίνει ότι το μοντέλο εκπαιδεύεται σε ΔΥΟ tasks ταυτόχρονα: το main 3-class clarity και το auxiliary 9-class evasion. Το σκεπτικό: τα δύο labels είναι σχετιζόμενα, οπότε ίσως η extra “εργασία” του να μάθει το λεπτομερές evasion να βοηθήσει τα clarity weights. Δούλεψε στο DistilBERT: **+0.020 mean** σε 3 seeds ( $0.6367 \pm 0.012$  vs baseline  $0.6167 \pm 0.021$ ). Επίσης μείωσε το seed variance, που σημαίνει ότι λειτούργησε σαν regularizer.

**5.3.2. Από DistilBERT σε BERT:** μικρή βελτίωση. Όταν ports-άρα την winning DistilBERT config (ml=256, lr 2e-5, 5 epochs) στο BERT, είδα best 0.6417 (seed=1). Ουσιαστικά τα ίδια. Ενδιαφέρον finding: το multi-task aux που δούλεψε στο DistilBERT δεν δούλεψε στο BERT – ήταν πρακτικά tied με το baseline (−0.002). Παρόμοιο pattern και στο DeBERTa, όπου το aux έβλαψε (−0.049).

Αυτό μου άρεσε σαν εύρημα: **auxiliary granularity helps weak representations** αλλά βλάπτει **strong ones**. Ένα DeBERTa έχει ήδη μάθει αυτές τις λεπτές διακρίσεις από το pre-training του – δεν χρειάζεται extra βοήθεια, αντίθετα η extra task το “μπερδεύει” γιατί η evasion taxonomy σπάει την Ambivalent κλάση σε υποκατηγορίες που το DeBERTa είχε ήδη ομαδοποιήσει σωστά.

**5.3.3. DeBERTa baseline:** το πραγματικό άλμα. Το DeBERTa-v3-base seed=42 χωρίς κανένα trick – απλά two-segment input, lr 2e-5, 3 epochs, ml=256, weight\_decay=0 – έδωσε **0.6962** macro-F1. Αυτό είναι +0.05 πάνω από το BERT, ένα τεράστιο jump για NLP standards.

Φυσικά μόνο seed=42 δεν είναι αξιόπιστο. Έτρεξα και seeds 0 και 1 για να δω variance: 0.6333 (seed=0), 0.6727 (seed=1). Δηλαδή το seed=42 ήταν *lucky*, αλλά ακόμα και ο μέσος όρος ( $0.6674 \pm 0.032$ ) είναι σαφώς καλύτερος από το BERT/DistilBERT.

Με weighted ensemble των 3 seeds έφτασα 0.7070 local val και 0.68 στο Kaggle (39η θέση τότε).

**5.3.4. Ιδέα 1: Feature tokens** μέσα στο κείμενο (απέτυχε). Από την HW1 είχα handcrafted features όπως: HEDGE\_HIGH (πολλά hedging words), OV\_LOW (λίγο overlap ερώτησης-απάντησης), LEN\_HIGH (μεγάλη απάντηση). Σκέφτηκα: “ωραία, ας τα βάλω σαν ειδικά tokens μέσα στο input, ας πούμε στην αρχή της απάντησης. Έτσι το DeBERTa θα “δει” explicit signals”.

Δηλαδή το input γινόταν κάτι σαν:

[CLS] question [SEP] [HEDGE\_HIGH] [OV\_LOW] [LEN\_HIGH] answer [SEP]

Αποτέλεσμα: **0.6191**, πολύ κάτω από το vanilla DeBERTa (0.6962).

Γιατί απέτυχε; Γιατί αυτά τα tokens ([HEDGE\_HIGH] κ.λπ.) είναι καινούρια, δεν υπάρχουν στο vocabulary του DeBERTa. Το μοντέλο πρέπει να μάθει embedding γι’ αυτά από το μηδέν, με μόλις 3000 παραδείγματα. Επιπλέον φέρονται σαν θόρυβος στην self-attention – το DeBERTa δεν ξέρει αν είναι σημαντικά ή όχι. Στην πρώτη εργασία αυτά τα features δούλευαν επειδή ο linear classifier δεν είχε άλλη επιλογή – χρησιμοποιούσε ό,τι του δίναμε explicit. Στο DeBERTa, η ίδια πληροφορία υπάρχει ήδη σιωπηρά μέσα στα contextual embeddings, και τα tokens απλώς βρωμίζουν τη ροή.

**Lesson learned:** το feature engineering της πρώτης εργασίας δεν γίνεται transferred

ως text tokens.

**5.3.5. Ιδέα 2: *Numeric side branch* (το *breakthrough*).** Ίδια ιδέα, σωστή υλοποίηση. Αντί να μπω στα **tokens**, έφτιαξα τα **features** σαν αριθμητικό **input** και τα έδωσα σε ξεχωριστό κανάλι. Αρχιτεκτονικά:

1. Το DeBERTa διαβάζει κανονικά το two-segment input και βγάζει το pooled vector (768 dims).
2. Παράλληλα, υπολογίζω 19 αριθμητικά features από το ζευγάρι Q-A: μήκος απάντησης (tokens, words, sentences), overlap ερώτησης-απάντησης, hedge density, negation count, evidence markers, κ.λπ.
3. Τα features κανονικοποιούνται με statistics μόνο από το *train split* (πολύ σημαντικό – αν χρησιμοποιούσα και το *val* ή το *test*, θα είχα leakage).
4. Περνάνε από ένα μικρό MLP (Linear → ReLU → Dropout → Linear) που τα μετατρέπει σε ένα 64-dim dense vector.
5. Concat του DeBERTa pooled vector με το feature vector → τελικό classifier → 3 logits.

Δηλαδή το DeBERTa βλέπει το text όπως πάντα, “καθαρό”, και τα handcrafted features μπαίνουν με ένα παράλληλο, completely separate pathway που πάει κατευθείαν στον τελικό classifier. Δεν παρενοχλούν την self-attention.

Αποτέλεσμα: **0.7007** (seed=42, single model). +0.005 πάνω από το vanilla DeBERTa, και κρίσιμα η Clear Reply F1 ανέβηκε αρκετά (από 0.611 σε 0.660). Με α-weighted 3-seed ensemble: 0.7057. Στο Kaggle αυτό ήταν το final submission: **0.70**.

**5.3.6. Ιδέα 3: *Word normalization* (ίσος συν).** Όπως εξήγησα στο preprocessing, αντικατέστησα αριθμούς, ποσοστά κ.λπ. με κανονικές αγγλικές λέξεις. Single model: 0.6965 (vs. 0.6951 του rerun raw). Πολύ μικρό gain στο single-model επίπεδο, αλλά διαφορετικά error patterns – αυτό είναι αξιόλογο για ensemble.

**5.3.7. Ιδέα 4: *Focused answer view*** (διαφορετικός τρόπος να “διαβάσει” την απάντηση). Αυτό είναι ένα από τα πειράματα που το όνομά του ίσως δεν σου λέει τίποτα, οπότε εξηγώ.

Πρόβλημα: πολλές πολιτικές απαντήσεις είναι τεράστιες (5+ προτάσεις) και γεμάτες αιτιολογήσεις, παρενθέσεις, αλλαγές θέματος. Το μοντέλο μπορεί να “χαθεί” στα 200 tokens και να μη συγκεντρωθεί στο σημαντικό μέρος της απάντησης.

**Idea:** αντί να δίνω ολόκληρη την απάντηση, δίνω ένα “focused view” – τις σημαντικότερες προτάσεις. Συγκεκριμένα, κρατάω:

- τις πρώτες 1-2 προτάσεις (συχνά εκεί ξεκινάει η ουσία της απάντησης),
- τις προτάσεις που έχουν το μεγαλύτερο **lexical overlap** με την ερώτηση (πιθανόν είναι αυτές που πιο άμεσα απαντάν),
- την τελευταία πρόταση (συμπερασματική).

Αυτό είναι το “**focused view**”. Αρχιτεκτονικά είναι ίδιο μοντέλο (DeBERTa + **numeric features**), απλά αλλάζει το **text input**.

Αποτέλεσμα μόνο του: 0.6796, χειρότερο από raw. Δηλαδή **focused view** σαν **standalone single model** δεν είναι καλύτερο, γιατί ξανά – η συνολική ρητορική ροή της απάντησης μετράει, δεν είναι μόνο τα “ουσιαστικά” σημεία.

Αλλά: σε **ensemble** βοήθησε. Γιατί; Γιατί κάνει διαφορετικά λάθη από το **full-view** μοντέλο. Όταν δύο μοντέλα έχουν παρόμοιο **score** αλλά διαφορετικά **error patterns**, ο μέσος όρος των **probabilities** ξεπερνάει το καθένα από τα δύο. Αυτή είναι η βασική θεωρία του **ensemble**: **complementary errors cancel out**.

**5.3.8. Ιδέα 5: Phase 18 alpha ensemble** (το καλύτερο **local val**). Βάλαμε σε **ensemble** τις τρεις διαφορετικές “**views**” του ίδιου DeBERTa+numfeat μοντέλου:

- raw full answer,
- word-normalized full answer,
- focused raw answer.

Έκανα **alpha sweep** πάνω στο **val set**: δοκιμάζω **weighted averages** των τριών **probability distributions** με βάρη που αθροίζουν σε 1, και κρατάω το συνδυασμό που μεγιστοποιεί το **macro-F1**. Best βάρη: 0.55 raw + 0.20 wordnorm + 0.25 focused → **0.7136 val macro-F1**.

Έκανα και ένα ακόμα βήμα, “**calibrated**” **ensemble**, όπου πολλαπλασίασα τις **final class probabilities** με σταθερές [0.70, 1.10, 0.70] για CR, AMB, CNR αντίστοιχα. Αυτό ονομάζεται **post-hoc calibration** ή απλά **threshold tuning**. Έδωσε 0.7154 **val** αλλά η **test distribution** ήταν **Ambivalent-heavy** (216 από 308 **test rows** ως AMB) που δείχνει ριψοκίνδυνο **submission**. Δεν το έκανα **submit** ως τελικό.

**5.3.9. Ιδέα 6: Dual-view architecture** (απέτυχε). Σκέφτηκα: ωραία, αν το **full** και το **focused** μοντέλο μαζί κάνουν καλό **ensemble**, γιατί να μην το βάλω μέσα σε ένα **joint** μοντέλο; Έφτιαξα το **dual-view classifier**:

- ένα DeBERTa backbone (shared weights),

- περνάει δύο φορές: μία με το full Q-A και μία με το focused Q-A,
- παίρνω τα δύο pooled vectors,
- τα concat-άρω μαζί με τα numeric features,
- τα δίνω σε classifier.

Theoretically αυτό θα έπρεπε να είναι καλύτερο από ensemble γιατί τα weights μαθαίνονται από κοινού. Πρακτικά: 0.6665 single, 0.6888 ensemble – χειρότερο από Phase 18 alpha (0.7136). Γιατί;

- Δύο passes του DeBERTa διπλασιάζουν τις παραμέτρους που “ενεργοποιούνται” στο gradient. Με μόλις 3000 train examples αυτό overfits πολύ γρήγορα.
- Έπρεπε να μειώσω το batch size σε 8 (από 16) για να χωράει στη GPU memory, που έκανε το optimization πιο noisy.
- Το βασικό πλεονέκτημα του ensemble (ανεξάρτητα εκπαιδευμένα μοντέλα με διαφορετικά errors) χάνεται όταν όλα τα weights είναι shared.

**Lesson learned:** σε μικρά datasets, joint architectures με shared backbone συχνά υστερούν από απλά ensembles. Το ensemble κερδίζει “δωρεάν diversity” που είναι πιο πολύτιμη από τα joint gradients.

**5.3.10. Ιδέα 7: Evasion-label stacking (απέτυχε).** Το dataset δίνει ένα δεύτερο label, το evasion\_label (9 κλάσεις, π.χ. “Explicit Reply”, “Dodging the Question”, “Claims Ignorance”), μόνο για το train set. Το test set έχει evasion\_label = “”, οπότε δεν μπορούσα να το χρησιμοποιήσω directly σαν feature.

Σκέφτηκα δεύτερη στρατηγική. **Stacking:** εκπαιδεύω ένα ξεχωριστό μοντέλο (TF-IDF + Logistic Regression) που προβλέπει το 9-κλάσεις evasion\_label, και τις probabilities αυτές τις δίνω σαν extra features στο numeric side branch του main DeBERTa. Πρόβλημα: αν εκπαιδεύσω και τα δύο μοντέλα στο ίδιο train, ο main classifier θα “δει” evasion probabilities που είναι in-sample (το evasion μοντέλο είδε αυτά τα ίδια rows), που είναι information leakage.

Λύση: **out-of-fold predictions.** Κάνω 5-fold cross validation για το evasion μοντέλο: σπάω το train σε 5 κομμάτια, εκπαιδεύω 5 evasion μοντέλα, και για κάθε row χρησιμοποιώ τις probabilities που προέβλεψε το μοντέλο που δεν είδε αυτό το row. Έτσι κάθε train row έχει “unseen” evasion probabilities. Για το val/test, χρησιμοποιώ ένα τελικό evasion μοντέλο εκπαιδευμένο σε όλο το train.

Αποτέλεσμα: 0.6503, καλύτερα από τίποτα αλλά αρκετά κάτω από το vanilla numfeat (0.7007). Γιατί απέτυχε; Πιθανόν γιατί το evasion taxonomy είναι σχετικό με το clarity, αλλά όχι ακριβώς ευθυγραμμισμένο. Π.χ. το “Clarification” στην evasion taxonomy μπορεί να αντιστοιχεί σε Clear Reply ή Ambivalent, ανάλογα. Άρα η evasion probability δεν είναι clean σήμα προς το clarity prediction – είναι correlated αλλά noisy.

**5.3.11. Ιδέα 8:** Περισσότερα *epochs* (απέτυχε). Φυσιολογική σκέψη: ίσως 3-4 epochs είναι λίγα. Δοκίμασα 6 epochs με την ίδια numfeat config. Αποτέλεσμα: 0.6876, χειρότερο. Δοκίμασα και label smoothing 0.05 για να δω αν αμβλύνει το overfit: 0.6810, ακόμα χειρότερα. **Conclusion:** το πρόβλημα δεν ήταν underfitting, ήταν το opposite – σε μικρό dataset το μοντέλο overfits γρήγορα. Έπρεπε να μείνω στα 3-4 epochs.

**5.3.12. Ιδέα 9: Longer context** (*max\_length=512*) στο **DeBERTa** (απέτυχε). Στο DistilBERT είχε δουλέψει το ml=512. Δοκίμασα το ίδιο στο DeBERTa: 0.6615 αντί 0.7007. Πιθανή εξήγηση: για το ml=512 χρειάζονται batch size 8 (αντί 16) λόγω GPU memory, οπότε λιγότερα gradient steps per epoch και πιο noisy optimization. Επίσης το DeBERTa είναι πιο ευαίσθητο σε longer sequences – η self-attention γίνεται  $O(n^2)$  και τα effective gradients μικραίνουν.

## 5.4. Hyper-parameter tuning

Τα hyperparameters που έπαιξα:

- **Learning rate:** 1e-5, 2e-5, 3e-5, 5e-5. Best: 2e-5 για BERT/DeBERTa, 3e-5 για DistilBERT (τα μικρότερα μοντέλα είναι πιο stable με μεγαλύτερο lr).
- **Number of epochs:** 3, 4, 5, 6. Best: 3-4. Πέρα από 4 το μοντέλο overfits.
- **Max length:** 128, 256, 512. Best για DeBERTa: 256. Best για DistilBERT: 512. Αυτό είναι interesting – για το DistilBERT, το extra context βοηθάει γιατί το μοντέλο είναι λιγότερο dense και χρειάζεται περισσότερο raw signal. Στο DeBERTa, όπου τα embeddings είναι ήδη πιο πυκνά, το ml=256 είναι αρκετό και το ml=512 βλάπτει λόγω optimization noise.
- **Weight decay:** 0.0, 0.01, 0.1. Best: 0.0. Ένα από τα πιο μη-διαισθητικά findings μου: η default του AdamW (0.01) έβλαπτε. Με wd=0.0 πήρα +0.01-0.02 σε όλα τα μοντέλα. Πιθανόν γιατί τα BERT-family μοντέλα έχουν ήδη internal regularization (dropout, layer norm), και το explicit weight decay προσθέτει υπερβολική penalty.
- **Seed:** 0, 1, 42. Το DeBERTa ήταν αρκετά seed-sensitive: 0.6333 / 0.6727 / 0.6962. Με numeric features: 0.6462 / 0.6684 / 0.7007. Αυτή η variance είναι ο βασικός λόγος που χρησιμοποιώ ensembles.
- **Batch size:** σταθερό 16 για ml=256, 8 για ml=512.

## 5.5. Optimization techniques

Χρησιμοποίησα **AdamW** [3], που είναι standard για fine-tuning Transformer μοντέλων. Σύντομα: είναι ο Adam optimizer αλλά με decoupled weight decay (το weight decay εφαρμόζεται απευθείας στα weights αντί μέσα στο gradient update). Αυτό αποδεικνύεται mathematically πιο σωστό για L2-regularization.

**Differential learning rate** για το DeBERTa: το backbone (το pretrained DeBERTa) παίρνει  $lr=2e-5$ , ενώ το head (classifier + numeric MLP) παίρνει  $10\times$  μεγαλύτερο  $lr$  ( $2e-4$ ). Λόγος: το backbone είναι ήδη καλά εκπαιδευμένο και θέλουμε λεπτή προσαρμογή, ενώ ο classifier ξεκινάει from scratch με random init και χρειάζεται γρήγορη μάθηση. Αυτό το trick διόρθωσε το αρχικό μου πρόβλημα όπου το DeBERTa “κόλλαγε” στο να προβλέπει μόνο Ambivalent.

**Linear scheduler with warmup:** για τα πρώτα 10% των steps, το  $lr$  αυξάνεται γραμμικά από 0 στο target  $lr$ . Μετά μειώνεται γραμμικά μέχρι 0 στο τέλος του training. Αυτό είναι standard practice και βοηθάει το optimization να είναι σταθερό στο αρχικό noisy phase.

**Gradient clipping** σε max norm 1.0. Παρ’ ότι σπάνια ενεργοποιείται, σε ένα από τα DeBERTa runs είδα grad norm 10.8 σε ένα update – χωρίς clipping θα μπορούσε να σπρώξει το μοντέλο σε bad region.

**Mixed precision (FP16)** για να επιταχυνθεί η εκπαίδευση και να μη μου τελειώσει η GPU memory στο Kaggle.

## 5.6. Evaluation

Η βασική metric για το assignment είναι το **macro-F1**. Παρ’ ότι το assignment ζητάει επίσης accuracy/precision/recall/F1, συνοπτικά (best DeBERTa+numfeat single, validation set):

| Class           | Precision | Recall | F1            | Support |
|-----------------|-----------|--------|---------------|---------|
| Clear Reply     | 0.663     | 0.657  | 0.660         | 105     |
| Ambivalent      | 0.783     | 0.767  | 0.775         | 202     |
| Clear Non-Reply | 0.625     | 0.714  | 0.667         | 35      |
| Macro avg       | 0.690     | 0.713  | <b>0.7007</b> | 342     |
| Accuracy        |           |        | <b>0.7281</b> | 342     |

Table 2: Detailed breakdown για το best DeBERTa+numeric-feature single model (seed=42).

**5.6.1. Γιατί macro-F1 και όχι plain accuracy.** Αν χρησιμοποιούσαμε accuracy: ένα μοντέλο που λέει συνέχεια Ambivalent θα είχε accuracy 60% (η Ambivalent είναι το 60% του dataset), και ίσως να φαινόταν αξιοπρεπές. Αλλά το macro-F1 του θα ήταν τραγικό γιατί η Clear Reply και η Clear Non-Reply F1 θα ήταν 0.

Το F1 είναι ο αρμονικός μέσος precision και recall:

$$F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Macro-F1 = average των F1 της κάθε κλάσης, ίσο βάρος. Αυτό αναγκάζει το μοντέλο να είναι καλό και στις τρεις κλάσεις, όχι μόνο στην επικρατούσα.

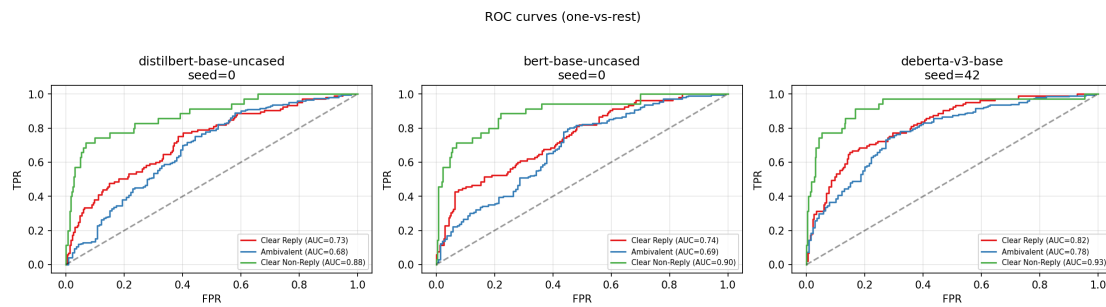


Figure 1: ROC curves για τα τρία βασικά μοντέλα (DistilBERT / BERT / DeBERTa) one-vs-rest. Η ROC curve δείχνει το true positive rate vs false positive rate σε διαφορετικά decision thresholds. Όσο πιο κοντά στην πάνω αριστερή γωνία, τόσο καλύτερο. Το DeBERTa έχει σαφώς τη μεγαλύτερη AUC (area under curve), επιβεβαιώνοντας ότι διαχωρίζει καλύτερα τις κλάσεις σε όλα τα thresholds.

### 5.6.2. ROC curve.

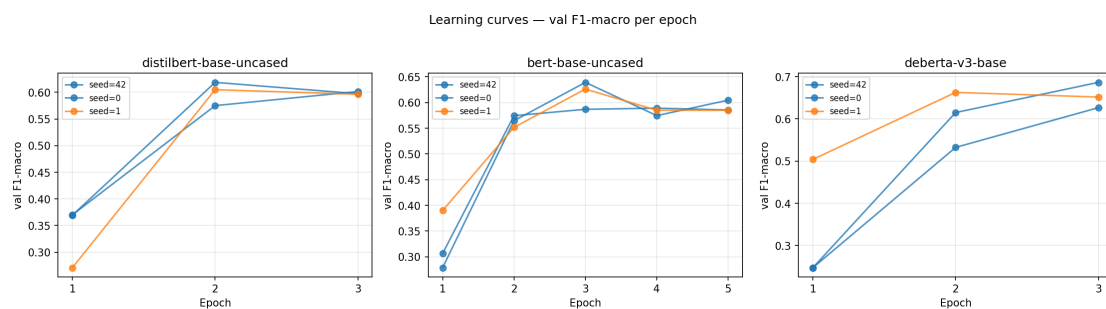


Figure 2: Learning curves: train loss και validation macro-F1 ανά epoch. Φαίνεται ότι μετά το epoch 3-4 το train loss συνεχίζει να πέφτει αλλά το validation σταθεροποιείται ή πέφτει – κλασικό σημάδι ότι το dataset είναι μικρό για επιθετικό fine-tuning. Γι' αυτό σταμάτησα τις προπονήσεις στα 3-4 epochs.

### 5.6.3. Learning curves.

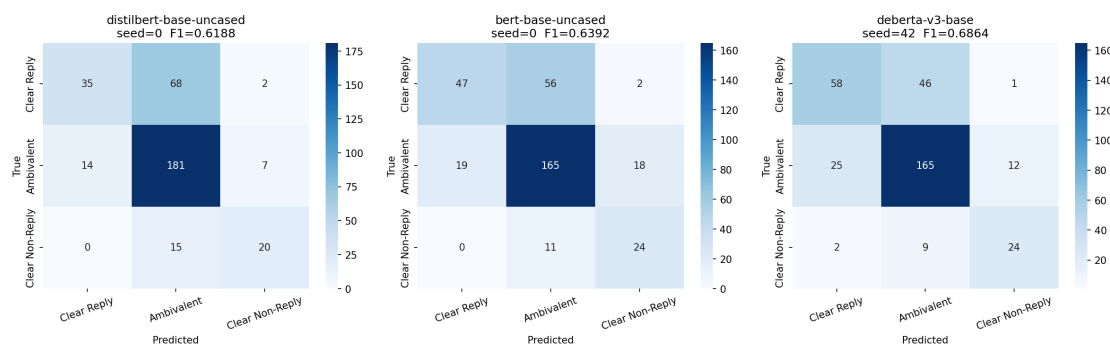


Figure 3: Confusion matrices για τα τρία μοντέλα. Σε όλα τα μοντέλα, το πιο συχνό λάθος είναι η μετακίνηση Clear Reply ή Clear Non-Reply προς την Ambivalent. Στο DeBERTa αυτό περιορίζεται αλλά δεν εξαφανίζεται.

### 5.6.4. Confusion matrices.

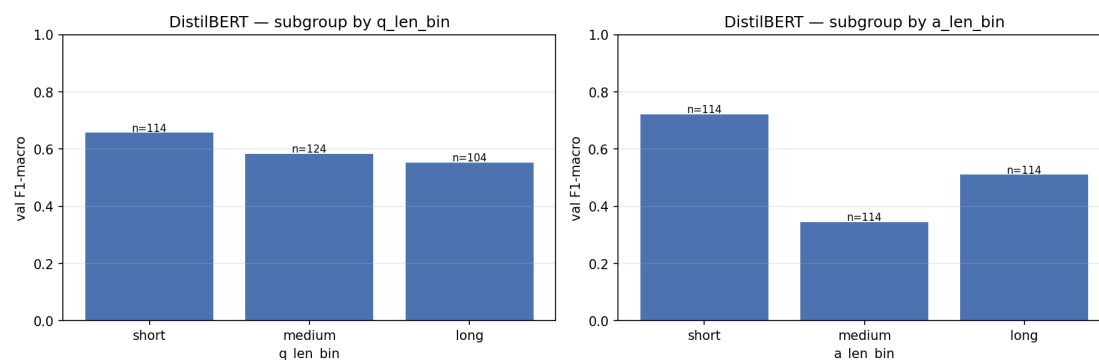


Figure 4: Subgroup analysis: macro-F1 ανά bin μήκους ερώτησης και απάντησης. Φαίνεται καθαρά ότι τα λάθη συγκεντρώνονται στις πιο μεγάλες απαντήσεις. Επίσης το DeBERTa διατηρεί καλύτερο score σε όλα τα bins σε σχέση με DistilBERT/BERT, αλλά και αυτό υποφέρει στις πολύ μεγάλες απαντήσεις.

### 5.7. Subgroup analysis

Η ανάλυση ανά subgroup έδειξε ένα ξεκάθαρο pattern: όσο μεγαλύτερη η απάντηση, τόσο χειρότερη η performance. Στις απαντήσεις που έχουν >150 tokens, το macro-F1 πέφτει κατά 5-8 percentage points σε όλα τα μοντέλα. Αυτό έχει νόημα: μεγάλες απαντήσεις είναι πιο πολύπλοκες ρητορικά – εκεί ο πολιτικός δίνει context, αιτιολογήσεις, παραδείγματα, και η “στάση” του μπορεί να αλλάζει μέσα στο ίδιο message.

Από την άλλη, το μήκος της ερώτησης είχε πολύ λιγότερη επίδραση. Αυτό είναι λογικό γιατί οι ερωτήσεις είναι σχετικά τυπικές (“what do you think about X”, “why X”) και δεν περιέχουν πολλή ambiguity.

Αυτό το finding είναι που με οδήγησε αρχικά στο focused-view πείραμα – “ίσως αν συμπίεσουμε τις μεγάλες απαντήσεις σε key sentences, βελτιωνόμαστε”. Όπως είδαμε, δεν

δούλεψε ως **single model**, αλλά βοήθησε ως **ensemble member**.

## 6. Detailed error analysis

### 6.1. Ποια κλάση είναι η πιο δύσκολη

Από το **confusion matrix** και τα **per-class F1 scores**: η **Clear Reply** είναι η δυσκολότερη ( $F1=0.660$ ), ακολουθεί η **Clear Non-Reply** ( $F1=0.667$ ), και η **Ambivalent** είναι η πιο εύκολη ( $F1=0.775$ ). Αυτό φαίνεται αντι-διαισθητικό μέχρι να σκεφτείς τη λογική:

- Η **Ambivalent** είναι το “default” στο οποίο πέφτει το μοντέλο όταν δεν είναι σίγουρο. Άρα έχει υψηλό **recall**.
- Η **Clear Reply** είναι το **most contested** – συχνά μια απάντηση είναι “almost clear” και μετακινείται στο **Ambivalent**.

### 6.2. Συγκεκριμένα παραδείγματα λαθών

Πάμε σε **concrete examples**. Παρακάτω είναι μερικά από τα **top-confidence** λάθη του **best** μοντέλου (**DistilBERT seed=0 baseline run**, αλλά τα **patterns** είναι ίδια και στο **DeBERTa**):

#### Example 1: Clear Reply → Ambivalent (confidence 0.827)

*Q: What is the greatest and most urgent threat when it comes to security that Barack Obama has to deal with?*

*A: The most urgent threat that he'll have to deal with, and other Presidents after him will have to deal with, is an attack [...]*

Το μοντέλο πρόβλεψε **Ambivalent** αλλά είναι **Clear Reply**. Γιατί έσφαλε; Ίσως γιατί η απάντηση ξεκινάει με **long restatement** της ερώτησης (“the most urgent threat that he'll have to deal with...”). Το μοντέλο μάλλον το διαβάζει σαν **hedging**.

#### Example 2: Clear Non-Reply → Ambivalent (confidence 0.814)

*Q: Specifically, have you seen any success in your diplomatic strategy so far?*

*A: Well, I think you know me well enough to know that I don't like talking about whether I see success or not in a case suc[h as this]...*

Πραγματικό **CNR** – ο πολιτικός λέει “δεν θα μιλήσω”. Αλλά το μοντέλο βλέπει το “**success or not**” και νομίζει ότι κάτι λέει. Δηλαδή το μοντέλο μπερδεύεται από **lexical overlap**: η απάντηση περιέχει τη λέξη της ερώτησης (“**success**”), που είναι κλασικό **marker** απάντησης – αλλά εδώ έχει αρνητική χρήση (“**I don't like talking about**”).

### Example 3: Clear Reply → Ambivalent (confidence 0.789)

Q: *What do you think we should do to help resolve this conflict between Russia and Georgia?*

A: *Precisely what we ought to do is help resolve the conflict and use our diplomats to convince people there is a better wa[y]...*

Καθαρή απάντηση, σχεδόν repeats την ερώτηση και δίνει δράση. Αλλά το μοντέλο διστάζει επειδή δεν δίνεται specific name/policy. Ίσως μάθηξε ότι “true Clear Reply” απαντήσεις περιέχουν specifics.

### 6.3. Patterns στα λάθη

Από τη διερεύνηση 100+ errors εντόπισα μερικά recurring patterns:

1. **Hedge phrases at the start** (“Well, I think...”, “Look, the truth is...”): το μοντέλο τα διαβάζει σαν evasion signals και μετακινεί το prediction προς Ambivalent, ακόμα κι όταν η ουσία της απάντησης είναι clear.
2. **Long answers με aside**: ο πολιτικός απαντάει σαφώς αλλά μετά αρχίζει 5 παραγράφους context. Το μοντέλο ζυγίζει υπερβολικά τον context.
3. **Lexical overlap traps**: η απάντηση περιέχει τις λέξεις της ερώτησης αλλά σε αρνητική χρήση. Π.χ. “you ask whether X is true. X is not the issue here.”
4. **Restatement-then-answer patterns**: η απάντηση επαναλαμβάνει την ερώτηση και μετά δίνει απάντηση. Το μοντέλο νομίζει ότι το restatement είναι hedge.
5. **Implicit non-replies**: ο πολιτικός δεν λέει “I won’t answer” αλλά αλλάζει θέμα. Αυτά είναι δύσκολα γιατί δεν έχουν explicit lexical signals – απαιτούν να καταλάβεις ότι η απάντηση δεν αντιστοιχεί σημασιολογικά στην ερώτηση.

### 6.4. Διαφέρουν τα στρονγέρ μοντέλα στα λάθη τους;

Σύγκρινα τα 90 errors του DeBERTa α-ensemble με τα errors του DistilBERT-aux και του BERT στο ίδιο val set. Από τα 90 DeBERTa errors:

- το DistilBERT-aux τα έβρισκε σωστά μόλις 21/90 (23%),
- το BERT τα έβρισκε 17/90 (19%).

Δηλαδή τα μοντέλα κάνουν παρόμοια λάθη. Όταν δοκίμασα cross-family ensemble (DeBERTa + DistilBERT + BERT), το α-grid πάντα κατέρρεε σε “μόνο DeBERTa”. Δεν υπάρχει αρκετή cross-family diversity για να αξιοποιηθεί.

Αυτό είναι ένα από τα πιο σημαντικά findings μου: τα **errors** είναι **structural**, όχι **random**. Δεν είναι ότι το ένα μοντέλο πρόσεχε X και το άλλο Y – και τα τρία αποτυγχάνουν στους ίδιους “γκρίζους” τύπους απαντήσεων. Άρα η αξία διαφορετικών μοντέλων εξαντλείται γρήγορα. Πιο χρήσιμη ήταν η **diversity** εντός του ίδιου DeBERTa (διαφορετικά **seeds**, διαφορετικές **input views**), που έδινε ίδιο τύπο αλλά διαφορετικά **noisy** επιλογές.

#### 6.5. Τι θα έκανε ένα “ιδανικό” μοντέλο για αυτό το **task**;

Ακόμα κι αν δεν θα μπορούσα να το υλοποιήσω εδώ, νομίζω ότι ένα μοντέλο που να ξεπερνούσε σαφώς το 0.75 macro-F1 θα χρειαζόταν:

- **Discourse-level reasoning:** όχι απλώς “τι λέει η απάντηση” αλλά “ποια στάση παίρνει η απάντηση απέναντι στην ερώτηση”. Αυτό είναι κάτι που τα current Transformer μαθαίνουν μερικώς αλλά όχι **explicitly**.
- **Stance-aware pretraining:** αν είχαμε ένα DeBERTa pretrained σε debate / interview κείμενα και αναλύσεις, θα είχε ήδη παρόμοια representations. Το general-domain pretraining δεν είναι αρκετά εξειδικευμένο.
- Πολλαπλό **data**. Με 3000 examples είναι δύσκολο να μάθεις λεπτές discourse-level διακρίσεις. 30,000 ή 100,000 πιθανότατα θα έλυναν τα περισσότερα από τα current errors.
- **Larger model:** deberta-v3-large αντί base. Δοκίμασα να το προσθέσω αλλά δεν χωρούσε στη GPU memory του Kaggle με ml=256, και θα έπρεπε να βάλω gradient accumulation κ.λπ. Δεν ολοκλήρωσα.

## 7. Results and Overall Analysis

### 7.1. Κύρια ευρήματα

Το βασικό story της εργασίας μπορεί να συμπυκνωθεί σε λίγες γραμμές:

1. Από TF-IDF/Logistic Regression (HW1, 0.6017) σε vanilla DeBERTa (0.6962) έχουμε +0.09 macro-F1 – το pretrained Transformer μοντέλο είναι σαφέστερα πιο κατάλληλο για discourse-level NLP tasks.
2. Μέσα στα τρία Transformers, το DeBERTa-v3-base νικάει τα DistilBERT/BERT καθαρά (+0.05 vs BERT). Δηλαδή η αρχιτεκτονική παίζει – δεν είναι μόνο θέμα μεγέθους.
3. Ο πιο ωφέλιμος συνδυασμός είναι DeBERTa + numeric side branch με handcrafted features. Όχι feature tokens μέσα στο κείμενο. Αυτή η διάκριση είναι κρίσιμη.
4. Πολλές “προφανείς” βελτιώσεις (περισσότερα epochs, longer context, richer features, dual-view, evasion stack) απέτυχαν. Με 3000 παραδείγματα τα δυνατά μοντέλα overfit γρήγορα.

5. **Ensemble** εντός του ίδιου DeBERTa (διαφορετικά seeds και input views) είναι πιο αποδοτικό από cross-family ensemble.

## 7.2. Best trial: αρχιτεκτονική

Το καλύτερο μου submitted Kaggle αποτέλεσμα (0.70 public macro-F1) προέρχεται από αυτή την αρχιτεκτονική:

1. **Tokenizer:** DeBERTa-v3 tokenizer, max\_length=256, two-segment Q-A input.
2. **Backbone:** microsoft/deberta-v3-base, frozen architecture, fine-tuned.
3. **Numeric features:** 19 features (lengths, overlaps, hedge counts, evidence markers, etc.) από custom Python code, normalized με statistics από μόνο το train split.
4. **Numeric MLP:** Linear (19 → 64) → ReLU → Dropout (0.2) → Linear (64 → 32).
5. **Fusion:** concat (DeBERTa pooled, 768) με numeric MLP output (32) → 800-dim vector.
6. **Classifier:** Linear (800 → 256) → ReLU → Dropout (0.2) → Linear (256 → 3).
7. **Loss:** standard cross-entropy.
8. **Training:** AdamW (lr=2e-5 backbone, 2e-4 head, eps=1e-6, wd=0), linear schedule με warmup, 4 epochs, batch size 16, mixed precision FP16.
9. **Ensemble:** 3 seeds (42, 0, 1),  $\alpha$ -weighted average των probabilities με weights [0.9, 0.0, 0.1].

Local val macro-F1 αυτής της εκδοχής: 0.7057. Public Kaggle: **0.70**.

Η ακόμα καλύτερη τοπική εκδοχή (Phase 18 alpha ensemble) έφτασε 0.7136 val, συνδυάζοντας raw + word-normalized + focused views, αλλά δεν την υπέβαλα ως τελικό γιατί ήθελα να κρατήσω το αξιόπιστο 0.70 leaderboard score.

## 7.3. Comparison with Assignment 1

Η πρώτη εργασία ήταν στο ίδιο dataset (CLARITY) αλλά με κλασικά models: TF-IDF + Logistic Regression / Random Forest / SVM. Best HW1 score: 0.6017 macro-F1. Διαφορές ως προς εδώ:

Τι ήταν διαφορετικό στο **HW1**:

- Έπρεπε να κάνω heavy feature engineering: hedge counts, overlap ratios, sentence structure, etc. – το μοντέλο δεν θα τα μάθαινε από μόνο του.

- **Cleaning** του κειμένου ήταν χρήσιμο (lemmatization, stopword removal κ.λπ.) γιατί τα bag-of-words μοντέλα ωφελούνται από vocabulary reduction.
- Δεν υπήρχε “contextual” πληροφορία – κάθε λέξη ήταν ανεξάρτητη.

Τι ήταν διαφορετικό εδώ:

- Το pretrained Transformer έχει ήδη μάθει γλωσσικά patterns. Δεν χρειάζεται να του εξηγήσω τι είναι hedging – το ξέρει από το internet.
- Heavy preprocessing βλάπτει αντί να βοηθάει.
- Feature engineering βοηθάει μόνο αν μπει σε σωστό σημείο της αρχιτεκτονικής (numeric side branch, όχι text tokens).

Σαν conclusion: τα δύο approaches δεν είναι εναλλακτικά – συμπληρώνονται. Το DeBERTa κερδίζει το semantic understanding, αλλά το feature engineering της HW1 προσθέτει explicit signals που το μοντέλο μπορεί να μη χρησιμοποιεί σταθερά. Το καλύτερο final αποτέλεσμα προέκυψε από συνδυασμό τους.

#### 7.4. Modifications που δοκίμασα και απέτυχαν – σύνοψη

Νομίζω είναι σημαντικό να αναφέρω συνοπτικά τις αποτυχίες, γιατί το assignment ζητάει discussion των modifications που δοκίμασα.

| Modification                   | Σύντομη εξήγηση                                          | Verdict                               |
|--------------------------------|----------------------------------------------------------|---------------------------------------|
| Class weights                  | Βάρη loss vs. class frequency                            | Χειρότερο                             |
| Focal loss                     | Down-weight εύκολων examples                             | Χειρότερο                             |
| cw + focal combo markers       | Συνδυασμός των δύο Special token πριν από αρνητικά verbs | Catastrophe NEG>NEG +0.008 (marginal) |
| Multi-task aux (DistilBERT)    | Co-learn evasion 9-class                                 | +0.020                                |
| Multi-task aux (BERT, DeBERTa) | Ίδιο αλλά σε strong backbones                            | Tied / Χειρότερο                      |
| Feature tokens μέσα στο text   | HW1 features σαν special tokens                          | Χειρότερο (-0.08)                     |
| Numeric side branch            | HW1 features σαν παράλληλο numeric input                 | <b>+0.005</b> (best single)           |
| Word normalization             | Years/money/dates → real words                           | +0.003 (small)                        |
| Focused answer view            | Crop στις key sentences                                  | Solo χειρότερο, ensemble +0.01        |
| Dual-view architecture         | Shared DeBERTa σε 2 views                                | Χειρότερο (overfitting)               |
| Evasion stack (OOF)            | Evasion proba ως extra features                          | Χειρότερο                             |
| More epochs (6)                | Επεκταμένο schedule                                      | Χειρότερο (overfit)                   |
| Label smoothing 0.05           | Soft targets                                             | Χειρότερο                             |
| max_length=512 (DeBERTa)       | Longer context                                           | Χειρότερο                             |
| 3-seed alpha ensemble          | Weighted avg of 3 seeds                                  | +0.005 (consistent)                   |
| Phase 18 multi-view alpha      | raw + wordnorm + focused                                 | <b>+0.013</b> (best ensemble)         |
| Calibrated alpha               | Class multipliers post-hoc                               | +0.002 (risky)                        |

Table 3: Σύνοψη των modifications. Δεξιά = best single DeBERTa+numfeat (0.7007) ως baseline για ensemble comparisons.

## 7.5. Τι θα έκανα διαφορετικά αν είχα περισσότερο χρόνο

- Πιο συστηματικό **feature ablation**: τώρα έχω 19 features σαν block. Δεν ξέρω ποιο από αυτά πραγματικά δουλεύει. Με leave-one-out θα μπορούσα να δω ποια είναι **useful** και ποια είναι **noise**.
- Περισσότερα **seeds** για το **Phase 18 ensemble**: τώρα είναι 3 seeds, με 5-7 ίσως να γινόταν πιο **stable**.

- **DeBERTa-v3-large**: αν είχα GPU memory, ίσως +0.02-0.03. Αλλά θα έπρεπε να ξανα-tune τα hyperparameters.
- **Pretraining σε domain-specific corpus**: μερικά intermediate steps σε political interview transcripts θα μπορούσαν να δώσουν stance-aware embeddings.
- **Calibration με Platt scaling ή temperature scaling** αντί χειρωνακτικών class multipliers.

## 8. Συμπεράσματα

Η εργασία αυτή έβαλε σε δοκιμή το πόσο μπορούν τα off-the-shelf pretrained Transformer μοντέλα να λύσουν ένα πραγματικό, “μεσαίας δυσκολίας” NLP task. Από τα 0.60 της κλασικής προσέγγισης φτάσαμε στο 0.70 public Kaggle και πάνω από 0.71 local val. Αυτό είναι ένα σχετικά καλό αποτέλεσμα δεδομένου ότι το dataset είναι μικρό ( 3000 παραδείγματα) και η Ambivalent κλάση είναι by design “γκρίζα”.

Από τα πειράματα έβγαλα μερικά πιο γενικά συμπεράσματα που νομίζω ότι αξίζουν:

- Στα μικρά datasets, τα strong pretrained μοντέλα κερδίζουν τα custom architectures. Λιγότερο είναι περισσότερο – vanilla DeBERTa + small numeric add-on > complex dual-view architecture.
- Feature engineering δεν είναι dead, απλώς πρέπει να μπει στη σωστή θέση του δικτύου.
- Auxiliary tasks helps weak backbones και βλάπτουν strong ones. Capacity-inverse finding.
- Ensembles εντός ίδιας οικογένειας μοντέλων είναι πιο αποδοτικά από cross-family όταν τα μοντέλα κάνουν structural errors.
- Όλα τα μοντέλα “λοξεύουν” προς το Ambivalent όταν είναι αβέβαια – αυτή είναι ίσως η βαρύτερη δυσκολία του task.

Στο τέλος της ημέρας, νομίζω ότι το πιο πολύτιμο μάθημα από αυτή την εργασία είναι ότι όλα αυτά τα modifications πρέπει να δοκιμάζονται systematically και με fair baselines. Αν είχα προσπαθήσει να πετάξω feature engineering και ensembles χωρίς να έχω σαφές vanilla baseline, δεν θα μπορούσα να ξέρω αν ή τι βοηθούσε. Σχεδόν τα μισά “smart” tricks που δοκίμασα τελικά δεν δούλεψαν – και αυτό είναι ένα από τα πιο σημαντικά findings.

## 9. Bibliography

### References

- [1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019.
- [2] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. In *International Conference on Learning Representations*, 2021.
- [3] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [4] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. In *NeurIPS EMC2 Workshop*, 2019.